

Week .3

Course Name : Problem Solving and Testing using java

Course Code : 10211CS227

Name : P.Varsha

Vtu No : 29196

1. Java Date and Time

2. Number of Days Between Two Dates

3. Day of the Year

4. Day of the Week

5. Java Priority Queue

6. Java ArrayList

7. Largest Number

8. Java Comparator 9. Sort

the People

1.Java Date and Time

PROGRAM:

```
import java.util.*;

publicclass Main {

    publicstatic String findDay(int month, int day, int year) {
        Calendar cal = Calendar.getInstance();

        cal.set(year, month - 1, day);

        String[] days = {
            "SUNDAY",
            "MONDAY",
            "TUESDAY",
            "WEDNESDAY",
            "THURSDAY",
            "FRIDAY",
            "SATURDAY"
        };

        return days[cal.get(Calendar.DAY_OF_WEEK) - 1];
    }
}
```

```
public static void main(String[] args) {  
  
    Scanner sc = new Scanner(System.in);  
  
    int month = sc.nextInt();  
    int day = sc.nextInt(); int  
    year = sc.nextInt();  
  
    System.out.println(findDay(month, day, year));  
  
    sc.close();  
}  
}
```

OUTPUT:

A screenshot of a Java IDE's console window. The window has a dark background with a toolbar at the top containing icons for a dropdown menu, a cursor, a window, a gear, and a trash can. The console output shows the date "21 05 2006" and the day of the week "WEDNESDAY" in white text. Below this, a green message states "...Program finished with exit code 0" and "Press ENTER to exit console." in green text. A small blue tab with the letter 'S' is visible on the left side of the console window.

```
21 05 2006  
WEDNESDAY  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

2.Number of Days Between Two Dates

PROGRAM:

```
import java.time.LocalDate; import
java.time.temporal.ChronoUnit;

classSolution {    public int daysBetweenDates(String
    date1, String date2) {
        LocalDate d1 = LocalDate.parse(date1); LocalDate
        d2 = LocalDate.parse(date2);

        return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
    }
}
```

OUTPUT:

Accepted Runtime: 4 ms

Case 1 Case 2

Input

date1 =
"2019-06-29"

date2 =
"2019-06-30"

Output

1

Expected

1

Accepted Runtime: 4 ms

Case 1 Case 2

Input

date1 =
"2020-01-15"

date2 =
"2019-12-31"

Output

15

Expected

15

3.Day of the Year

PROGRAM:

```
import java.time.LocalDate;

classSolution {    public int
    dayOfYear(String date) { LocalDate d =
    LocalDate.parse(date); return
    d.getDayOfYear();
    }
}
```

OUTPUT:

Accepted Runtime: 4 ms

✓ Case 1 ✓ Case 2

Input

date =
"2019-01-09"

Output

9

Expected

9

Accepted Runtime: 4 ms

✓ Case 1 ✓ Case 2

Input

date =
"2019-02-10"

Output

41

Expected

41

4.Day of the Week

PROGRAM:

```
import java.time.LocalDate;
```

```
classSolution { public String dayOfTheWeek(int day, int  
    month, int year) {
```

```
    LocalDate date = LocalDate.of(year, month, day);
```

```
    String dayName = date.getDayOfWeek().toString();
```

```
    return dayName.charAt(0) + dayName.substring(1).toLowerCase();
```

```
}
```

```
}
```

OUTPUT:

Accepted Runtime: 5 ms

Case 1 Case 2 Case 3

Input

day =
31

month =
8

year =
2019

Output

"Saturday"

Expected

"Saturday"

Accepted Runtime: 5 ms

Case 1 Case 2 Case 3

Input

day =
18

month =
7

year =
1999

Output

"Sunday"

Expected

"Sunday"

Accepted Runtime: 5 ms

Case 1 Case 2 Case 3

Input

day =
15

month =
8

year =
1993

Output

"Sunday"

Expected

"Sunday"

5.Java Priority Queue

PROGRAM:

```
import java.util.*;
```

```
classStudent {
```

```
    private int id;
```

```
    private String name; private
```

```
    double cgpa;
```

```
    public Student(int id, String name, double cgpa) { this.id
```

```
        = id;
```

```
        this.name = name;
```

```
        this.cgpa = cgpa;
```

```
    }
```

```
    public int getID() { return
```

```
        id;
```

```
    }
```

```
    public String getName() { return
```

```
        name;
```

```
    }
```

```

    public double getCGPA() { return
        cgpa;
    }
}

```

```

class Priorities {

```

```

    public List<Student> getStudents(List<String> events) {

```

```

        PriorityQueue<Student> pq = new PriorityQueue<>(
            (a,b)-> { if(a.getCGPA() != b.getCGPA()) { return
                Double.compare(b.getCGPA(), a.getCGPA());
            }

```

```

            int nameCompare = a.getName().compareTo(b.getName());

```

```

            if(nameCompare != 0) {
                return nameCompare;
            }

```

```

            return Integer.compare(a.getID(), b.getID());
        }
    };

```

```

    for(String event : events) {

```

```

    if(event.startsWith("ENTER")) {
        String[] parts = event.split(" ");

        String name = parts[1];

        double cgpa = Double.parseDouble(parts[2]);
        int id = Integer.parseInt(parts[3]);

        pq.add(new Student(id, name, cgpa));

    }elseif(event.equals("SERVED"))
        { if(!pq.isEmpty()) { pq.poll();
            }
        }
    }

    List<Student> result = new ArrayList<>();

    while(!pq.isEmpty()) { result.add(pq.poll());
    }

    return result;
}
}

```

```

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in); int n = sc.nextInt();
        sc.nextLine();

        List<String> events = new ArrayList<>();

        for(int i=0; i < n; i++) { events.add(sc.nextLine());
        }

        Priorities priorities = new Priorities();
        List<Student> students = priorities.getStudents(events);

        if(students.isEmpty()) {
            System.out.println("EMPTY");
        } else { for(Student student :
            students) {
                System.out.println(student.getName());
            }
        }

        sc.close ();
    }
}

```

OUTPUT:

```
5  
SURYA 99  
VAMSHI 90  
SIDDHU 100  
HARSHA 91  
SIVA 89  
EMPTY  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

6.Java ArrayList

PROGRAM:

```
import java.util.*;
```

```
public class Main { public static void
    main(String[] args) {
        Scanner sc = new Scanner(System.
in);

        int n = sc.nextInt();

        ArrayList<ArrayList<Integer>> list = new ArrayList<>();

        //Read the lines
        for(int i=0; i<n; i++) {
            int size = sc.nextInt();

            ArrayList<Integer> row = new ArrayList<>();

            for(int j=0; j<size; j++) {
                row.add(sc.nextInt());
            }

            list.add(row);
        }

        //Number of queries
        int q = sc.nextInt();
```

```

for(int i=0;i<q;i++) {
    int x=sc.nextInt();
    int y=sc.nextInt();    //x and y are
    1-based

    if(x>= 1 && x <= list.size() && y>= 1
        && y <= list.get(x - 1).size()) {

        System.out.println(list.get(x - 1).get(y - 1));

    }else {
        System.out.println("ERROR!");
    }
}

sc.close();
}
}

```

OUTPUT:

 Console

 I/O

 AI Agent

5

5 41 77 74 22 44

1 12

4 37 34 36 52

0

3 20 22 33

5

1 3

74

3 4

52

7.Largest Number

PROGRAM :

```
import java.util.*;

classSolution {    publicString

    largestNumber(int[] nums) { String[] arr =

    new String[nums.length];

    for(int i = 0; i < nums.length; i++) { arr[i]

        = String.valueOf(nums[i]);

    }

    Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));

    if(arr[0].equals("0")) {

        return "0";

    }

    StringBuilder result = new StringBuilder();

    for(String s : arr) { result.append(s);

    }
```

```
        return result.toString();  
  
    }  
}
```

OUTPUT :

Accepted Runtime: 1 ms

✓ Case 1

✓ Case 2

Input

nums =
[10,2]

Output

"210"

Expected

"210"

Accepted Runtime: 1 ms

✓ Case 1

✓ Case 2

Input

nums =
[3,30,34,5,9]

Output

"9534330"

Expected

"9534330"

8.Java Comparator PROGRAM

:

```
import java.util.*;
```

```
classPlayer {
```

```
    Stringname;
```

```
    intscore;
```

```
    Player(String name, int score) {
```

```
        this.name = name;
```

```
        this.score = score;
```

```
    }
```

```
}
```

```
classChecker implements Comparator<Player> {
```

```
    @Override publicint compare(Player
```

```
    a, Player b) {
```

```
//Sort by decreasing score
if(a.score != b.score) {
    return b.score - a.score;
}

//Ifscores are same, sort alphabetically
return a.name.compareTo(b.name);
}

}
```

```
public class Main {

    publicstaticvoidmain(String[] args) {

        Scannersc=newScanner(System.in);

        int n= sc.nextInt();
        Player[]players=new Player[n];

        for(inti=0;i<n;i++) {
            Stringname=sc.next();
            intscore=sc.nextInt();

            players[i]=newPlayer(name, score);
        }

    }

}
```

```
Arrays.sort(players,new Checker());

for(Playerp:players) {
    System.out.println(p.name + " " + p.score);
}

sc.close();
}
```

OUTPUT :



```
4
surya 500
vamshi 600
siva 700
siddhu 900
siddhu 900
siva 700
vamshi 600
surya 500
```

9.Sort the People

PROGRAM :

```
import java.util.*; classSolution { publicString[]sortPeople(String[]
names, int[] heights) { Integer[]index=new Integer[names.length];

    for(inti=0;i<names.length; i++) {
        index[i] =i;
    }

    Arrays.sort(index,(a, b) -> heights[b] - heights[a]);

    String[]result=new String[names.length];

    for(inti=0;i<names.length; i++) {
        result[i]=names[index[i]];
    }

    return result;
}
}
```

OUTPUT:

Accepted Runtime: 1 ms

✓ Case 1

✓ Case 2

Input

names =
["Mary", "John", "Emma"]

heights =
[180, 165, 170]

Output

["Mary", "Emma", "John"]

Expected

["Mary", "Emma", "John"]

Accepted Runtime: 1 ms

✓ Case 1

✓ Case 2

Input

names =
["Alice", "Bob", "Bob"]

heights =
[155, 185, 150]

Output

["Bob", "Alice", "Bob"]

Expected

["Bob", "Alice", "Bob"]